

```
#include <stdio.h>
```

```
int main() {  
    int n = 5;
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) printf("*");  
        printf("\n");  
    }  
    return 0;  
}
```

```
int main() {  
    int n = 5;
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1)
```

```
                print f("x");
```

```
            else
```

```
                print (" ");
```

```
        } return 0;
```

```
int main() {  
    int n = 5;  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= i; j++) {  
            printf(" ");  
        }  
        printf("\n");  
    }  
}
```

```
return 0;  
}
```

②

```
int main() {  
    int n = 5;  
    for (int i = n; i >= 1; i--) {  
        for (int j = 1; j <= i; j++) {  
            printf(" ");  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

```
int main() {  
    int n = 5;  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            printf(" ");  
        }  
        for (int j = 1; j <= (2 * i - 1); j++) {  
            printf(" ");  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

C Programming – Star Pattern Programs

C Programming

Week 1 Module – Star Pattern Programs

All 8 Patterns | n = 5

Pattern 1 : Square

SOURCE CODE

```
#include <stdio.h>

// Pattern 1: Solid Square - prints n rows of n stars each
int main() {
    int n = 5;

    // Outer loop controls rows, inner loop controls columns
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*****
*****
*****
*****
```

Pattern 2 : Hollow Square

SOURCE CODE

```
#include <stdio.h>

// Pattern 2: Hollow Square - stars only on the border
int main() {
    int n = 5;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Print star only on first/last row or first/last column
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1)
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*  *
*  *
*  *
*  *
*****
```

Pattern 3 : Triangle

SOURCE CODE

```
#include <stdio.h>

// Pattern 3: Right-Angled Triangle - increasing stars per row
int main() {
    int n = 5;

    // Each row i prints exactly i stars
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
**
***
****
*****
```

Pattern 4 : Inverted Triangle

SOURCE CODE

```
#include <stdio.h>

// Pattern 4: Inverted Triangle - decreasing stars from n down to 1
int main() {
    int n = 5;

    // Each row i prints i stars, counting from n down to 1
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
****
***
**
*
```

Pattern 5 : Pyramid

SOURCE CODE

```
#include <stdio.h>

// Pattern 5: Pyramid - centered with leading spaces and odd-count stars
int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        // Print (n - i) leading spaces to center each row
        for (int j = i; j < n; j++) {
            printf(" ");
        }
        // Print (2*i - 1) stars for the current row
        for (int j = 1; j <= (2 * i - 1); j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
***
*****
*****
*****
```


Pattern 6 : Inverted Pyramid

SOURCE CODE

```
#include <stdio.h>

// Pattern 6: Inverted Pyramid - widest row first, tapering down to one star
int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        // Print (n - i) leading spaces to align each row
        for (int j = i; j < n; j++) {
            printf(" ");
        }
        // Print (2*i - 1) stars for the current row
        for (int j = 1; j <= (2 * i - 1); j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*****
*****
***
*
```

Pattern 7 : Half Diamond

SOURCE CODE

```
#include <stdio.h>

// Pattern 7: Half Diamond - ascending triangle joined with descending triangle
int main() {
    int n = 5;

    // Upper half: stars increase from 1 to n
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    // Lower half: stars decrease from n-1 down to 1
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
**
***
****
*****
****
***
**
*
```

Pattern 8 : Diamond

SOURCE CODE

```
#include <stdio.h>

// Pattern 8: Diamond - full symmetric diamond using pyramid + inverted pyramid
int main() {
    int n = 5;

    // Upper half: pyramid growing from 1 star to (2*n-1) stars
    for (int i = 1; i <= n; i++) {
        for (int j = i; j < n; j++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("*");
        printf("\n");
    }

    // Lower half: inverted pyramid shrinking from (2*(n-1)-1) stars to 1
    for (int i = n - 1; i >= 1; i--) {
        for (int j = i; j < n; j++) printf(" ");
        for (int j = 1; j <= (2 * i - 1); j++) printf("*");
        printf("\n");
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
***
*****
*****
*****
*****
*****
***
*
```

C++ Programming – Star Pattern Programs

C++ Programming

Week 1 Module – Star Pattern Programs

All 8 Patterns | n = 5

Pattern 1 : Square

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 1: Solid Square - prints n rows of n stars each
int main() {
    int n = 5;

    // Outer loop controls rows, inner loop controls columns
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*****
*****
*****
*****
```

Pattern 2 : Hollow Square

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 2: Hollow Square - stars only on the border
int main() {
    int n = 5;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Print star only on first/last row or first/last column
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1)
                cout << "*";
            else
                cout << " ";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*  *
*  *
*  *
*  *
*****
```

Pattern 3 : Triangle

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 3: Right-Angled Triangle - increasing stars per row
int main() {
    int n = 5;

    // Each row i prints exactly i stars
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
**
***
****
*****
```

Pattern 4 : Inverted Triangle

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 4: Inverted Triangle - decreasing stars from n down to 1
int main() {
    int n = 5;

    // Each row i prints i stars, counting from n down to 1
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
****
***
**
*
```

Pattern 5 : Pyramid

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 5: Pyramid - centered with leading spaces and odd-count stars
int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        // Print (n - i) leading spaces to center each row
        for (int j = i; j < n; j++) {
            cout << " ";
        }
        // Print (2*i - 1) stars for the current row
        for (int j = 1; j <= (2 * i - 1); j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
***
*****
*****
*****
```


Pattern 6 : Inverted Pyramid

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 6: Inverted Pyramid - widest row first, tapering down to one star
int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        // Print (n - i) leading spaces to align each row
        for (int j = i; j < n; j++) {
            cout << " ";
        }
        // Print (2*i - 1) stars for the current row
        for (int j = 1; j <= (2 * i - 1); j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*****
*****
*****
***
*
```

Pattern 7 : Half Diamond

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 7: Half Diamond - ascending triangle joined with descending triangle
int main() {
    int n = 5;

    // Upper half: stars increase from 1 to n
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    // Lower half: stars decrease from n-1 down to 1
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
**
***
****
*****
****
***
**
*
```

Pattern 8 : Diamond

SOURCE CODE

```
#include <iostream>
using namespace std;

// Pattern 8: Diamond - full symmetric diamond using pyramid + inverted pyramid
int main() {
    int n = 5;

    // Upper half: pyramid growing from 1 star to (2*n-1) stars
    for (int i = 1; i <= n; i++) {
        for (int j = i; j < n; j++) cout << " ";
        for (int j = 1; j <= (2 * i - 1); j++) cout << "*";
        cout << "\n";
    }

    // Lower half: inverted pyramid shrinking from (2*(n-1)-1) stars to 1
    for (int i = n - 1; i >= 1; i--) {
        for (int j = i; j < n; j++) cout << " ";
        for (int j = 1; j <= (2 * i - 1); j++) cout << "*";
        cout << "\n";
    }

    return 0;
}
```

PROGRAM OUTPUT

```
*
***
*****
*****
*****
*****
*****
***
*
```